

Documento de Arquitectura

Elección CAP: Consistencia y Tolerancia a Particiones (CP)

Sistema distribuido *LibreMercado*

Integrantes

Joselyn Montaña

Nicolás Malebrán

Asignatura

Sistemas Distribuidos

Profesor

Juan Torres

1. Contexto

LibreMercado se implementa como un sistema distribuido con **tres nodos independientes de base de datos**, uno por sucursal (Norte, Centro y Sur). Cada nodo mantiene su propio inventario (**stock**) y registra las ventas y compras realizadas en esa sucursal. El catálogo de productos, clientes/usuarios y proveedores está **replicado** en los tres nodos. Una sola aplicación PHP centraliza el acceso: consulta los tres nodos vía AJAX para mostrar el stock consolidado en una página única, y ejecuta las operaciones de venta, compra, traspaso de inventario y administración del catálogo contra el o los nodos correspondientes mediante PDO.

2. Elección: Consistencia + Tolerancia a Particiones (CP)

Ante una partición de red (un nodo de sucursal deja de responder), el sistema prioriza la **consistencia** del inventario y del catálogo por sobre la **disponibilidad total de escritura** en el nodo afectado. En la práctica:

- **Lecturas:** se mantiene disponibilidad parcial. La página sigue funcionando y muestra el stock de los nodos accesibles; el nodo caído se marca como “no disponible” (N/D) en vez de bloquear toda la vista.
- **Ventas:** si el nodo de la sucursal que vende no responde, la venta se rechaza por completo (HTTP 503). No se permite registrar una venta “a medias” ni descontar stock de forma optimista.
- **Compras / reabastecimiento:** es una transacción ACID local al nodo receptor; si éste no responde, la compra se cancela íntegramente.
- **Trasposos entre sucursales (2PC):** requieren que *ambos* nodos (origen y destino) estén disponibles y confirmen la operación.
- **CRUD de catálogo replicado (productos, usuarios, proveedores):** requiere que los *tres* nodos estén disponibles para aplicar el cambio.

Two-Phase Commit manual (trasposos)

1. **Prepare:** se abre `beginTransaction()` en ambos nodos y se ejecutan los `UPDATE` de stock (resta en origen, suma en destino), sin confirmar.
2. **Commit:** solo si ambas preparaciones fueron exitosas se ejecuta `commit()` en ambos nodos.
3. **Rollback:** si cualquiera de los dos nodos falla (caído, sin stock suficiente, error de red), se ejecuta `rollback()` en el o los nodos que llegaron a abrir transacción, y la operación se cancela íntegramente.

Commit atómico de N nodos (catálogo replicado)

Para productos, usuarios y proveedores se generaliza el mismo principio: `MultiNodo::ejecutar()` abre transacción en los tres nodos antes de ejecutar cualquier cambio. Si no logra conectarse o iniciar transacción en *alguno* de los tres, aborta sin tocar ningún nodo. Solo si los tres confirman la operación se hace `commit` en los tres. Los IDs de nuevos registros se generan de forma centralizada (`MultiNodo::siguienteId()`) para que el mismo registro tenga el mismo identificador en los tres nodos.

3. Justificación frente a fallo de conexión entre nodos

Si se priorizara **disponibilidad** (modelo AP) en este dominio, un traspaso, una venta o una edición de catálogo podrían confirmarse en un nodo y no en el otro durante una partición, generando estados como: stock descontado en la sucursal de origen pero nunca acreditado en destino, una venta registrada sin respaldo real de inventario, o un producto con distintos precios/IDs según el nodo consultado.

En un sistema de comercio donde el stock representa inventario físico real y el catálogo debe verse igual desde cualquier sucursal, esa inconsistencia tiene un costo operativo y de confianza directo: sobreventa, descuadres contables, y datos de catálogo divergentes entre sucursales.

Por eso se acepta el trade-off de **CP**: durante una partición, ciertas operaciones de escritura en el nodo afectado (o que dependan de él) quedarán **temporalmente no disponibles**, devolviendo un mensaje claro al usuario (por ejemplo, “*Sucursal Centro no está disponible. Operación cancelada*”), pero el estado del inventario y del catálogo permanece siempre correcto y verificable en los nodos sanos. Ningún cambio queda “a medio aplicar”.

4. Resumen de comportamiento por escenario

Escenario	Comportamiento del sistema
Los tres nodos disponibles	CRUD, ventas, compras y traspasos operan con normalidad.
Un nodo caído, sin operaciones sobre él	El resto del sistema opera con normalidad; la columna/estado de ese nodo muestra “N/D” o aparece marcado como caído.
Venta en sucursal cuyo nodo está caído	Operación rechazada (HTTP 503), sin cambios en ningún nodo.
Compra (reabastecimiento) en nodo caído	Operación rechazada, sin cambios en ningún nodo.
Traspaso donde origen o destino está caído	Rollback completo (2PC), sin cambios en ningún nodo.
Stock insuficiente en origen del traspaso	Rollback completo (2PC), sin cambios en ningún nodo.
CRUD de producto/usuario/proveedor con algún nodo caído	Operación rechazada antes de iniciar cualquier cambio (MultiNodo), sin escrituras parciales en los nodos sanos.

5. Conclusión

La arquitectura CP es coherente con la naturaleza del dominio (control de inventario físico distribuido y catálogo compartido entre sucursales), donde una inconsistencia entre nodos es más costosa que una indisponibilidad temporal y acotada de las operaciones de escritura durante una falla de red. El sistema implementa este modelo de forma concreta mediante transacciones PDO locales (ACID por nodo), Two-Phase Commit manual para operaciones de dos nodos (traspasos), y un commit atómico de los tres nodos para el catálogo replicado (**MultiNodo**), garantizando en todos los casos que, ante una partición, el sistema falla de forma segura: *rechaza la operación completa antes que aplicarla parcialmente*.